

Aula 5 - Rotas e Navegação (Angular Router)

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Uso de diretivas (`*ngIf` , `*ngFor`), pipes e serviços.
 - Dúvidas sobre o projeto de listagem de usuários.

2. Introdução ao Angular Router (25 min)

Conceitos Fundamentais

- Rotas: Mapeamento entre URLs e componentes.
- Configuração Básica:

typescript

```
const routes: Routes = [  
  { path: 'users', component: UserListComponent },  
  { path: 'users/:id', component: UserDetailComponent },  
  { path: '', redirectTo: '/users', pathMatch: 'full' }  
];
```

- Diretivas do Router:
 - `<router-outlet>` : Área de renderização dos componentes.
 - `routerLink` : Navegação declarativa em templates.

Recursos Avançados

- Parâmetros de Rota: Acessar via `ActivatedRoute` .
- Guards: `CanActivate` (proteção de rotas).
- Lazy Loading: Carregamento sob demanda de módulos.

3. Serviço `Router` (20 min)

- Navegação Programática:

typescript

```
constructor(private router: Router) {}  
this.router.navigate(['/users', userId]);
```

- **Passando Dados:**

- Via parâmetros (/users/:id).
- Via estado (this.router.navigate(['/details'], { state: { user } })).

Parte Prática (3 horas)

Atividade: Sistema de Usuários com Rotas

Passo 1: Configuração Inicial (45 min)

1. Gerar componentes:

bash

```
ng generate component user-detail  
ng generate component user-form
```

2. Configurar rotas (app-routing.module.ts):

typescript

```
const routes: Routes = [  
  { path: 'users', component: UserListComponent },  
  { path: 'users/new', component: UserFormComponent },  
  { path: 'users/:id', component: UserDetailComponent },  
  { path: 'users/:id/edit', component: UserFormComponent },  
  { path: '', redirectTo: '/users', pathMatch: 'full' }  
];
```

3. Adicionar <router-outlet> ao app.component.html :

html

```
<nav>  
  <a routerLink="/users">Lista</a>  
  <a routerLink="/users/new">Novo Usuário</a>  
</nav>  
<router-outlet></router-outlet>
```

Passo 2: Implementar UserDetailComponent (1h)

1. Recuperar parâmetro da rota (user-detail.component.ts):

typescript

```
export class UserDetailsComponent implements OnInit {
  user: any;

  constructor(
    private route: ActivatedRoute,
    private userService: UserService
  ) {}

  ngOnInit() {
    const userId = this.route.snapshot.params['id'];
    this.userService.getUser(userId).subscribe(user => this.user = user);
  }
}
```

2. Template (user-detail.component.html):

html

```
<div *ngIf="user">
  <h2>{{ user.name }}</h2>
  <p>Email: {{ user.email }}</p>
  <button routerLink="/users/{{user.id}}/edit">Editar</button>
</div>
```

Passo 3: Implementar UserFormComponent (1h)

1. Lógica para criar/editar (user-form.component.ts):

typescript

```
export class UserFormComponent implements OnInit {
  user: any = { name: '', email: '' };

  constructor(
    private route: ActivatedRoute,
    private router: Router,
    private userService: UserService
  ) {}

  ngOnInit() {
    const userId = this.route.snapshot.params['id'];
    if (userId) {
      this.userService.getUser(userId).subscribe(user => this.user = user);
    }
  }

  onSubmit() {
    if (this.user.id) {
      this.userService.updateUser(this.user).subscribe(() => {

```

```

        this.router.navigate(['/users', this.user.id]);
    });
} else {
    this.userService.addUser(this.user).subscribe(() => {
        this.router.navigate(['/users']);
    });
}
}
}

```

2. Template com formulário reativo (user-form.component.html):

html

```

<form (ngSubmit)="onSubmit()">
  <input [(ngModel)]="user.name" name="name" placeholder="Nome">
  <input [(ngModel)]="user.email" name="email" placeholder="Email">
  <button type="submit">Salvar</button>
</form>

```

Passo 4: Atualizar UserListComponent (30 min)

1. Adicionar links para detalhes (user-list.component.html):

html

```

<div *ngFor="let user of users">
  <a [routerLink]="['/users', user.id]">{{ user.name }}</a>
</div>

```

2. Estilização básica:

css

```

nav a { margin-right: 15px; }

```

Tarefa para Casa

1. Proteger rotas com AuthGuard :

- o Criar um guarda que redirecione para /login se o usuário não estiver autenticado.

2. Implementar CanDeactivate :

- o Alertar o usuário se houver alterações não salvas no formulário.

Próxima Aula

- **Tópico:** Formulários Reativos Avançados.
- **Atividade prática:** Validação customizada e formulários aninhados.

Resultado Esperado

- Aplicação com navegação entre:
 - Lista de usuários (`/users`).
 - Detalhes do usuário (`/users/:id`).
 - Formulário de criação/edição (`/users/new` e `/users/:id/edit`).

plaintext

Fluxo de Navegação:

Lista → Detalhes → Edição → Lista

<https://angular.io/generated/images/guide/router/route-animation.gif> (*Fluxo de navegação no Angular Router*) 🚀